ERL Software Reference and Trigger-Study Configuration

11 October 2026

Contents

Scope and software organization **1**

Core ERL functionality **2**

Input features and direct conversion **5**

Study scoring and configuration **8**

Data preparation and quantitative output **10**

Usage **11**

Operational limitations **13**

Scope and software organization

The existential rule-list (ERL) model discovers short, executable conditions that distinguish two supplied datasets. In this study:

- **Signal input:** an anomalous-region mixture, potentially dominated by ordinary minbias events.
- **Background input:** the minbias reference.
- **Optimization target:** separation of these inputs, not classification using their underlying process labels.

ERL returns ranked, potentially overlapping rules, not a sequential first-match classifier. Anomaly detection and hardware trigger implementation are outside the software's discovery workflow.

Component	Responsibility
Core ERL	Array/schema validation, rule generation, supplied-score evaluation, built-in pruning, prediction and OR combinations
Direct converter	Raw Parquet to object/event tensors, feature definitions and provenance
Study runner	PT loading, input-mode selection, penalized Gini, pruning configuration and rate-unit conversion
Reporting tools	Logs, structured JSON reports and acceptance plots

The principal study entry point is `casual_tests/run_ERL_Gini_nested.py`. It supports single-object or particle-pair inputs, event-only and combined discovery. Final pruning is model built-in; physics-specific rate units and assumptions remain external.

Core ERL functionality

Inputs and schemas

`ExistentialRuleList` accepts the following arrays **for each class**:

Input	Shape	`fit` arguments
Objects	`[N,S,P]`	`signal`, `background`
Object presence	`[N,S]`	`signal_slot_mask`, `background_slot_mask`
Events	`[N,F]`	`signal_event_features`, `background_event_features`
Optional weights	`[N]`	`signal_weight`, `background_weight`

- Configure object observables, event observables, or both. Event-only fitting omits objects and masks; no dummy tensor is required.
- Sample counts may differ between classes. Object slot/feature dimensions and feature definitions must agree. Combined inputs must have aligned rows.
- Each `ObservableSpec` declares a name and kind: continuous, ordinal, categorical or Boolean. Categories use consecutive integer codes starting at zero; ordinal values are exact integers; Booleans are 0 or 1.
- Valid measurements must be finite. Values excluded by object padding or a false validity guard are ignored during measurement validation.
- Weights must be finite and nonnegative; omitted weights are one per event. Internal numeric arrays use NumPy float64.
- A presence mask excludes padded object slots. Without one, the core treats every slot as present; it does not infer padding from zeros.
- A configured event schema requires its complete event matrix, even if a learned rule uses only some columns.

Rule semantics and validity

A combined rule can express:

```
event: met > 100 AND
(EXISTS object: type_id in {muons} AND pt > 20)
```

- All object conditions in a rule apply to the **same object**.
- `minimum_count=k` requires at least `k` objects satisfying that entire object conjunction. Pure event rules impose no object-count requirement.
- Event conditions restrict accepted events without changing which slots satisfy the object conditions.
- Mixed-type or asymmetric multi-object triggers are naturally expressed with event counts, leading/subleading features or a dedicated pair tensor. A same-single-object type conjunction cannot mean "one muon and one electron."
- A particle pair can occupy one object slot. All member and pair predicates then apply to that **same pair**, not independently chosen particles. Counting pair slots does not count distinct particles: pairs can share members.
- Conditions within a rule use AND. `RuleUnion` uses OR across rules, evaluating each rule's object-count requirement independently.

Declare an automatic measurement guard in the schema:

```
guarded_schema = (
    ObservableSpec("mass", "continuous", validity_column="mass_valid"),
    ObservableSpec("mass_valid", "boolean"),
)
```

Every mass comparison then also requires `mass_valid == True` on the same object. Event measurements use the same mechanism in event scope.

- Invalid values do not generate thresholds and cannot pass upper or lower cuts. Valid physical zeros remain usable.

- Guards apply in generation, prediction, slot matching, unions and logical pruning. They do **not** consume rule depth.
- The flag must be a separate, unguarded Boolean column in the same scope. Guard chains/cycles are not supported.
- `allow_validity_cuts=False` is the default: linked flags are guard-only, not independent True/False candidates. Explicit opt-in permits standalone flag cuts without weakening measurement guards.
- Guarding depends on schema links, not a `_valid` suffix or placeholder value. Ordinary Boolean features and meaningful object-absence counts remain eligible.

Beam search and object priority

Generation proceeds through the following stages:

1. Start from all events and present objects.
2. Extend each beam prefix with one object or event condition.
3. Test numeric midpoints using `>` and `<=` by default; subsample ordered thresholds when the threshold budget is exceeded. Test singleton categorical/Boolean membership conditions.
4. Evaluate whole-rule accepted yields against the original class totals. Reject nonfinite scores and insufficient signal-input acceptance.
5. Rank by the supplied larger-is-better score; retain up to the beam width, optionally applying beam similarity.
6. Pool retained prefixes from all depths, rank them and apply the candidate cap.

Key behavior:

- Each explicit cut consumes one depth, regardless of scope. Numeric thresholds are recalculated from measurements eligible under the prefix.
- Short prefixes remain final candidates; a rule need not reach maximum depth.
- Exact deduplication tracks event selections, surviving object slots and object-priority progress. Equal event acceptance alone is insufficient.
- Search is approximate. A wider beam can preserve useful extensions; increasing the final candidate cap cannot recover prefixes lost earlier.

Ordered, repeated and directed object cuts

Optional `object_cut_priority=["type_id", "pt"]` makes the next object cut consume the next list position:

Prefix state	Eligible next object feature
No type cut	`type_id` only
Type used, PT unused	`pt` only
Both used	Any enabled object feature, including refinements

- Event cuts stay unrestricted and do not advance object priority.
- Entries can repeat. `["type_id", "pt", "pt"]` requires two PT cuts after the type cut; using PT once does not consume both positions.
- A numerical entry can restrict direction to `>`, `>=`, `<` or `<=`. Python accepts `"pt:>"` or `("pt", ">")`; plain names leave direction free.
- For example, `["type_id", ("pt", ">"), ("pt", "<=")]` requires a lower bound followed by an upper bound on the same object. Bounds are optimized from data; they are not fixed by the priority list.
- Incomplete priority rules remain final candidates and, by default, can be retained.
- If a required feature has no usable cut, that prefix cannot extend through another object feature; event extensions remain possible.
- Feature names must belong to the enabled object schema. Directions apply to numerical features, not categorical membership. An empty list or `None` disables priority.
- Priority imposes no per-type beam quotas or exemptions from acceptance limits.
- Directed interval bounds still take two search steps. Their first bound must survive the beam; this is not joint interval optimization.

Optional completion requirement

`prune_candidates(require_complete_priority=True)` retains only rules consuming every configured priority entry, including repetitions and directions.

- Completion is checked against the original ordered cut history, not simplified display predicates. Event cuts and automatic validity guards do not consume entries.
- Incomplete prefixes remain available during beam search and in `candidate_rules_`. Completion filtering occurs before all other final-selection stages, so an incomplete rule cannot act as a pruning witness against a completed one.
- The option is off by default. An empty priority list is already complete. If no complete candidate survives search, final selection is empty; no fallback or automatic depth/beam increase is made.

Progress and mask storage

- `fit(progress_callback=callback)` emits search-stage dictionaries. The runner's `--progress` displays depth, processed beam parents, candidate cuts, beam filtering and elapsed time on stderr. It does not estimate a whole-fit ETA or cover final pruning.
- Search masks are bit-packed; only the current parent's masks are unpacked. Non-retained expansions are released after beam selection, and completed-layer results keep event selections rather than unnecessary slot masks.
- Packing reduces mask payload about eightfold without changing search semantics. Input arrays and expanded candidates still use memory; beam width is not a hard limit on the number of temporary expansions. Public rules expose Boolean masks.

Similarity and final pruning

Similarity uses Jaccard overlap, intersection divided by union; two empty selections have overlap one.

- **Beam similarity:** compares signal/reference event selections and surviving slots, only between prefixes at the same object-priority stage.
- **Final similarity:** compares signal/reference event selections only.
- Both class comparisons must satisfy their thresholds. Higher thresholds prune less aggressively; candidates are considered score-first.
- Overlapping prefixes can still support different later cuts. Aggressive beam similarity may remove useful refinements before they can be constructed.

After `fit`, call `prune_candidates`. Its selection order is:

```
candidate pool
  -> priority-completion eligibility (optional)
  -> reference-acceptance eligibility
  -> normalization and equivalence merging
  -> stricter-rule improvement filtering
  -> final similarity filtering
```

All stages are configurable. The core preserves original candidates and IDs; selected rules and decisions are stored separately.

Logical pruning is enabled by a finite, signed `stricter_score_improvement`; `None` disables it.

- Normalize repeated bounds and category constraints separately in object/event scope; merge equivalent rules, preferring shorter rules, then score and ID.
- A stricter rule must satisfy `score(stricter) - score(looser) > threshold`, with `1e-12` roundoff tolerance. Tighter bounds, additional AND conditions and minimum-count ordering are supported.
- Positive thresholds require improvement; zero requires a strictly better score. Negative thresholds allow bounded score loss: `-0.01` permits a drop smaller than 0.01 relative to every applicable retained looser rule.
- Allowing score loss does not change ranking or disable equivalence, similarity, completion or acceptance filtering. It can preserve more-specific descriptions that are not the highest-coverage selections.
- Looser rules are considered first; retained results are finally score-ranked.
- Containment is proved from predicates, not coincident training selections. Semantic implications between different features are not inferred.
- Original steps and scores are unchanged; normalized cuts provide a separate simplified representation.
- Eligibility comes first: incomplete or over-budget rules cannot prune eligible candidates. Final similarity can still remove looser or stricter alternatives.

`max_reference_acceptance` is an optional fraction in `[0,1]`, applied to each rule's weighted training acceptance. It is not a combined OR/menu limit.

Model outputs and defaults

The model exposes:

- `candidate_rules_`: capped candidate pool, ordered by score.
- `selected_rules_`, `pruning_result_`: final selection and pruning audit.
- `get_rule`, `evaluate_rule`, `build_union`: retrieval, evaluation and OR.
- `input_mode_`: fitted input mode.

Rules retain conditions, depth, scores and class counts/weighted yields at each step. Use `predict` for event acceptance and `slot_mask` for matching objects; the latter does not aggregate the minimum object count. Event-only prediction uses `predict(event_features=...)`.

TPR/FPR refer to weighted acceptance of the supplied signal/background inputs. Reported training precision is not genuine-signal purity when the signal input is a mixture.

Core defaults versus the principal study runner are listed together in the configuration section below. Fitting generates candidates; final pruning requires its own call.

Built-in scoring alternatives are:

- `TPRFPRRuleScore` and `TargetFPRRuleScore`.
- `PurityCoverageRuleScore` and `FScoreRuleScore`.
- `SignificanceRuleScore` and `LikelihoodRatioRuleScore`.

A custom callable may replace them.

Input features and direct conversion

Converter contract and saved tensors

Entry points:

- Function: `raw_event_conversion.convert.convert_raw_parquet_to_pt`.
- CLI: `python -m raw_event_conversion`.

The converter reads flat raw Parquet directly, without regionization, shuffling or rejecting empty-object events. It accepts local filenames/globs and remote file URLs, including EOS through optional XRootD dependencies. It does not enumerate an EOS directory. `--max-events` caps saved rows, not the amount transferred from each opened file.

Default sources are `L1T_Electron`, `L1T_MuonTight`, `L1T_PhotonTight`, `L1T_JetPuppiAK4` and `L1T_PUPPIMET`. Required missing columns cause an error; there is no fallback to non-PUPPI jets/MET.

PT entry	Default content
`obj`	float32 `[N,22,20]`; no MET object
`obj_columns`, `obj_feature_schema`	Ordered object definitions, types, applicability and guards
`pair_obj`, `pair_slot_mask`	Optional float32 `[N,231,68]` pairs and Boolean `[N,231]` presence
Pair metadata	Optional column names, schema and source-object index pairs
`event_features`	float64 `[N,193]`
`event_feature_columns`, `event_feature_schema`	Ordered names, kinds, units, sources and formulas
`label`	Process label for diagnostics, not the discovery target
`native_event_info`	Optional `[N,10]` diagnostic block; not runner input
`pf`	Zero compatibility placeholder, not actual PF candidates
Provenance	Source-file and source-row indices; neighboring `.sources.json`

These shapes describe converter defaults, not fixed ERL requirements. Add `--with-pair-tensor` / `include_pair_tensor=True` to save the separate pair tensor; existing object and event tensors are unchanged. Conversion schemas/settings must agree between datasets. Saved feature-to-flag links determine runtime guards. New columns require regenerated tensors.

Object features

Objects are PT-ranked within type and capped at:

- Jets: 10 slots, type ID 0.
- Muons: 4 slots, type ID 1.
- Electrons: 4 slots, type ID 2.
- Photons: 4 slots, type ID 3.

The 20 columns comprise `pt, eta, phi, type_id`, eight extra properties, and their eight validity flags:

Extra property	Applicable objects	Definition
`abs_d0`	Muons/electrons	Absolute stored D0, mm
`isolation_rho_corr`	Muons/electrons/photons	Stored corrected isolation
`mass`	Jets	Stored mass, GeV
`n_charged`, `n_neutrals`	Jets	Constituent counts
`charged_fraction`	Jets	Charged count / total constituent count
`ehad_over_eem`	Electrons/photons	Hadronic/electromagnetic energy ratio
`charge`	Jets/muons/electrons	Stored charge

Padding and invalid values:

- Unused slots are zero-filled and excluded by the runner's `pt > 0` mask.
- Missing/inapplicable extra properties are zero with a false validity flag. Real measured zeros can have true flags.
- Missing/nonfinite object eta/phi is zero-filled **without dedicated guards**.
- Counts and charge must be integers. Mass, counts and energy ratios are nonnegative; finite negative corrected isolation is preserved.

Particle-pair features

The optional pair tensor uses all unordered combinations of the 22 retained object slots: 231 possible pairs, excluding self-pairs and reversed duplicates.

- Each slot stores two distinct objects. Member order is increasing type ID, then decreasing PT within a type; equal-PT ties retain producer order.

- The 68 columns comprise 44 member columns (the 20 object fields plus eta/phi guards for each member), and 12 derived quantities with 12 validity flags.
- Derived quantities include minimum/scalar-sum/vector-sum PT, PT balance, invariant mass, absolute eta/phi separations, delta R, maximum isolation, minimum absolute D0, opposite-charge and opposite-eta-hemisphere indicators.
- Pair mass uses raw coordinates, valid measured jet masses, fixed lepton masses and zero photon mass. Required missing measurements invalidate the result.
- `pair_slot_mask` records both members' presence; schema-linked flags protect derived and copied measurements. Padded values never replace valid pairs.
- Pairs use capped objects; event summaries still use full stored collections. Pair slots scale quadratically with the object count and can substantially increase fitting time and memory.

The runner's `--pair-tensor` selects `pair_obj` instead of `obj`; events remain enabled unless `--object-only` is supplied. See `raw_event_conversion/PAIRS.md` for the schema and ordering details.

Event features

Event features use full stored collections **before object-slot caps**. They add no eta, isolation, overlap-removal or quality selection beyond the producer collection. Total PT is a collection-level scalar sum, not overlap-cleaned HT or the producer ScalarHT.

Family	Columns including flags	Main quantities
Multiplicities	20	Four types at PT $>$ 0 and four thresholds each
Per-type PT sums	4	Scalar sum of finite positive PT
Total activity and MET	5	Total PT, MET, total-PT/MET
Leading/subleading PT	8	Two PT ranks per type
Same-type PT ratios	16	Leading/sum and subleading/leading
Ranked absolute eta	16	Two ranks per type
Same-type pair angles	24	Leading/subleading separations
Cross-type pair angles	36	Leading objects for six type pairs
Object–MET angles	10	Five wrapped azimuthal angles
Charged-PF PT selections	8	Counts and PT sums
Charged-PF impact selections	38	Binned/cumulative counts and sums; invalid counts
Ranked lepton D0	8	Leading/subleading electrons and muons
Total	**193**	44 ordinal, 92 continuous, 57 Boolean

Omitting MET features gives 179 columns. Threshold overrides or added features can change the width.

Multiplicity thresholds. Each type includes finite PT $>$ 0. Additional strict thresholds, in GeV, are:

- Muons: 5, 10, 20, 40.
- Electrons and photons: 10, 20, 30, 50.
- Jets: 20, 30, 50, 100.

Names use `n_<type>_pt<threshold>`; `sum_pt_<type>` sums every finite positive PT. `sum_pt_all_objects` adds the four sums.

Ranks and ratios.

- `leading_pt_<type>`, `subleading_pt_<type>`: largest and second-largest finite positive PT, zero if absent.
- `leading_pt_fraction_<type>`: leading PT / same-type scalar PT sum.
- `pt_balance_<type>`: **subleading/leading**, requiring two objects.
- `met`: stored PUPPI MET; `sum_pt_over_met` requires valid MET $>$ 0.
- Undefined ratios and coordinates have linked validity flags. Counts, sums and ranked PT retain meaningful zero/absence values.

Angular definitions. PT ranking uses producer order to break ties:

$$|\Delta\eta| = |\eta_1 - \eta_2|, \qquad |\Delta\phi| = |\operatorname{atan2}(\sin(\phi_1 - \phi_2), \cos(\phi_1 - \phi_2))|,$$
$$\Delta R = \sqrt{(\Delta\eta)^2 + (\Delta\phi)^2}.$$

- Same-type features compare leading/subleading objects; cross-type features compare the leading objects of each type, not the closest pair.
- Six cross-type pairs cover all combinations of electrons, muons, photons and jets.
- MET angles compare each leading object type with MET: `delta_phi_leading_<type>_met` for jet, photon, electron and muon. `min_delta_phi_jets_met` also gives the minimum across all eligible jets.
- Angles are in radians with explicit guards. MET direction requires finite positive MET and finite phi. The minimum jet angle uses all eligible jets, including soft ones. Invalid coordinates do not promote a lower-ranked object.

Charged PF and displacement.

- Use unweighted `L1T_PFPart_PT` and finite nonzero charge, with no extra vertex association or eta/quality selection.
- PT windows: > 2, (2,5), (2,10), > 10 GeV; each gives count and scalar PT sum. Windows overlap and PT boundaries are strict.
- For charged PF PT > 2 GeV, absolute D0 bins in mm are [0,0.1), [0.1,0.5), [0.5,1), [1,5), [5,10), [10,infinity).
- Absolute significance bins are [0,2), [2,3), [3,5), [5,10), [10,20), [20,infinity).
- Cumulative thresholds: D0 > 0.5, 1, 5 mm; significance > 3, 5, 10.
- Significance is `abs(D0)/ErrorD0`, requiring finite D0, positive finite uncertainty and finite ratio. Invalid measurements are excluded; two counts record invalid D0/significance.
- Ranked electron/muon D0 has validity flags. Feature names containing `dxy` refer to stored producer D0, not a recomputed primary-vertex quantity.

The production caveats in `raw_event_conversion/IMPACT_PARAMETERS.md` apply: the audited chain lacks executed impact-parameter smearing and the checked shard had zero PF ErrorD0. Displacement or availability cuts alone do not demonstrate realistic displaced-trigger discrimination.

Study scoring and configuration

Gini and reference penalty

The external adapter `gini_rule_score` computes weighted Gini for each whole conjunction versus its complement on the **original samples**:

$$I(p) = 2p(1-p), \qquad G = I_{\text{parent}} - \frac{W_{\text{pass}}}{W_{\text{parent}}} I_{\text{pass}} - \frac{W_{\text{fail}}}{W_{\text{parent}}} I_{\text{fail}}.$$

Here `p` is the anomalous-input fraction in a region and `W` its total training weight. `ReferencePenalizedGini` supplies:

$$\text{score} = G - \lambda a_R,$$

where `a_R` is full-reference acceptance and `--reference-penalty` controls finite nonnegative `lambda`.

- Zero penalty recovers Gini; 1% reference acceptance contributes `0.01*lambda`.
- The penalty affects beam search and ranking. Negative scores remain eligible.
- Larger penalties favor reference rejection but can reduce coverage or amplify rare fluctuations; they do not impose a hard rate limit.
- Runners use unit event weights and do not automatically rebalance class sizes.
- Gini can favor either enrichment direction. Sharing its formula with EDT does not make the searches identical: EDT splits a current node, while ERL scores whole conjunctions and retains a finite beam.

Defaults and CLI controls

Setting	Core ERL default	Flexible runner
Maximum rule depth	4	4
Beam width	100	40
Candidate cap	1000	300
Numeric threshold cap	64	32
Minimum matching count	1	1
Minimum signal-input acceptance	0	0.01
Object-cut priority	Off	Off
Required priority completion	Off when pruning	Off
Progress output	No callback	Off
Standalone linked validity cuts	Off	Off
Beam similarity	On	Off
Beam class similarity thresholds	0.95 / 0.95	0.1 / 0.1
Scorer	Caller-supplied	Gini minus reference penalty; lambda = 1
Final similarity	On when pruning is called	Off
Logical improvement threshold	Disabled	0; strict improvement required

The flexible runner enables both tensors by default; rate filtering is off unless requested. Search depth, width, caps and minimum acceptance are constructor settings in the script, not CLI flags.

Inputs and feature selection

- `--signal`, `--background`: anomalous-mixture and reference PT files.
- `--event-only` / `--no-object-features`: disable objects.
- `--object-only` / `--no-event-features`: disable event features.
- `--pair-tensor` / `--use-pair-tensor`: load saved pairs instead of single objects.
- `--basic-object-features`: retain PT, eta, phi and type per member, plus linked coordinate guards when required by the pair schema; omit derived pair features.
- `--object-cut-priority FEATURE[:DIRECTION] [...]`: ordered cut entries; repeats and numerical directions are supported. Quote operators in Bash.
- `--require-complete-priority`: exclude incomplete candidates before final pruning.
- `--allow-validity-cuts`: permit standalone linked flag cuts.
- `--progress`: display search progress on stderr.

Both modalities cannot be disabled. Basic-object mode and priority require objects; priority names must belong to the enabled object schema.

Scoring and selection

- `--reference-penalty`: default 1.
- `--stricter-score-improvement`: signed score-difference threshold, default 0; larger values prune more strict subsets, negative values allow bounded loss.
- `--beam-similarity-filter`, `--final-similarity-filter`: independent opt-ins.
- `--signal-similarity-threshold`, `--background-similarity-threshold`: default 0.1 each when similarity is enabled.
- `--max-trigger-rate-khz`: optional per-rule ceiling.
- `--input-rate-mhz`: default 40, used for rate estimates.
- `--report`: JSON report path; `--overwrite-report` permits replacement.

Enabled tensors must be nonempty, schema-compatible and aligned with the diagnostic `label` vector. Disabled tensors are ignored. Expanded object tensors require their saved schema. The runner reads available columns from metadata rather than assuming the converter's default dimensions.

Trigger-rate translation

The study passes the following acceptance ceiling to core pruning:

$$a_{R,\max} = \frac{R_{\max}\,[\mathrm{kHz}]}{1000\,R_{\mathrm{input}}\,[\mathrm{MHz}]}.$$

- 40 kHz at 40 MHz gives 0.001, or 0.1%.
- Rate filtering runs after search but before logical/similarity final selection.
- The estimate assumes the reference represents the input stream.
- Limits are per rule; combined OR rate and uncertainty must be evaluated separately. Logical score pruning is not a rate/coverage Pareto filter.

Data preparation and quantitative output

Mixture construction

`scripts/make_reference_anomalous.py` mixes compatible background/signal PT files.

- Defaults: 2,000 reference background events; a mixture of 1,000 different background events plus 100 signal events; seed 42.
- Sampling is without replacement and the mixture is shuffled.
- Background rows in reference and mixture are disjoint. Independence also requires independent source events without duplicated input events.
- Aligned tensors, labels, schemas and provenance are preserved; shapes, dtypes, keys and non-tensor metadata must be compatible.
- Conversion and mixing overwrite with a warning by default; `--no-overwrite` protects outputs. Reports instead require explicit overwrite.

The demonstration mixture is assembled explicitly, not selected by an upstream anomaly detector. A real anomaly-selected background component can differ from inclusive minbias; discovered rules may describe that selection difference.

Reports and plots

Reports contain **JSON in a `.txt` file**, loadable with `json.load`:

- Input sizes/paths, feature schemas, enabled modes and full configuration.
- Candidate IDs, raw/normalized predicates, effective depth and score.
- Mixture/reference accepted counts and fractions; raw Gini and penalty terms.
- Prefix histories, automatic guards, remaining priority entries and completion status.
- Selected tensor key, pair schema, progress/completion configuration and removal counts.
- Final retention, pruning reason, witness ID and optional rate estimate.
- Stored-process diagnostics, which never enter the score.

All capped candidates are saved, including final-pruned and incomplete priority rules. Prefixes removed during beam search are not included. Displayed normalized conditions can be reordered; prefix histories preserve the generation order. Completion exclusions have status `incomplete_object_priority` and no witness. `object_cut_priority_complete` and `object_cut_priority_remaining` describe each candidate; `summary.incomplete_priority_rules_removed` counts excluded prefixes.

Ranking/statistic	Meaning
`rank` / printed `global_rank`	Score order in the full capped pool
`pruned_rank`	Score order among retained rules
Section rank	Position within that printed section
Acceptance ratio	Mixture/reference acceptance; null if reference acceptance is zero

Use `retained_after_final_pruning` and `pruning.status` to interpret selection. Report metrics use raw counts; they match weighted acceptance in the unit-weight runner, not necessarily in custom weighted studies.

`scripts/plot_acceptance_report.py` plots reference acceptance horizontally and mixture acceptance vertically, with rule paths on the right. It filters the requested subset and preserves report order; it does not rerank by purity. `scripts/plot_all_reports.sh` provides batch plotting.

Usage

Core API

Prepare matching `ObservableSpec` schemas, arrays and Boolean presence masks before calling the core. For imports from the checkout, set `PYTHONPATH=src`.

```
from set_based_models import ExistentialRuleList, TPRFPRRuleScore

model = ExistentialRuleList(
    object_schema, event_observables=event_schema,
    scoring_function=TPRFPRRuleScore(), min_signal_efficiency=0.01,
    object_cut_priority=["type_id", ("pt", ">")])
model.fit(
    mixture_objects, reference_objects,
    signal_slot_mask=mixture_mask, background_slot_mask=reference_mask,
    signal_event_features=mixture_events,
    background_event_features=reference_events)
selected = model.prune_candidates(
    stricter_score_improvement=0.01, max_reference_acceptance=0.001,
    similarity_filter=False, require_complete_priority=True)
menu = model.build_union([rule.rule_id for rule in selected])
accepted = menu.predict(validation_objects, slot_mask=validation_mask,
                        event_features=validation_events)
```

- **Event-only:** construct with `event_observables=event_schema` and no object schema/priority; fit only the two event-feature keyword arguments.
- **Object-only:** omit event observables and event-feature arrays.

Conversion, mixture, discovery and plotting

Run from the repository root in an environment with the required dependencies. Replace raw paths with actual filenames/globs; labels below are diagnostic IDs.

```
python -m raw_event_conversion \
  --input raw/minbias/*.parquet \
  --output data/minbias_raw.pt --label 1 --max-events 10000

python -m raw_event_conversion \
  --input raw/signal/*.parquet \
  --output data/signal_raw.pt --label 2 --max-events 10000

python scripts/make_reference_anomalous.py \
  --background data/minbias_raw.pt \
  --signal data/signal_raw.pt \
  --output-dir data/example_mixture

python casual_tests/run_ERL_Gini_nested.py \
  --signal data/example_mixture/anomalous.pt \
  --background data/example_mixture/reference.pt \
  --report reports/ERL_example_combined.txt \
  --object-cut-priority type_id 'pt:>' --require-complete-priority \
  --reference-penalty 10 --stricter-score-improvement 0.01 \
  --max-trigger-rate-khz 40 --input-rate-mhz 40

python scripts/plot_acceptance_report.py \
  reports/ERL_example_combined.txt \
  --output reports/ERL_example_combined.png
```

- Add `--object-only` to restrict discovery to objects.
- For event-only, add `--event-only` and remove object priority.
- To enable both similarity stages, add `--beam-similarity-filter --final-similarity-filter` and choose thresholds.
- To use pairs, convert both classes with `--with-pair-tensor`, then add `--pair-tensor`; configure priorities using member names such as `type_id_1`.
- Completion is optional; omit `--require-complete-priority` to retain useful incomplete prefixes. Add `--progress` for search-stage feedback.
- Penalty 10, improvement 0.01 and priority are example settings, not defaults or universally recommended values. Use distinct report names for comparisons.

Operational limitations

- **Search:** finite beams, threshold budgets and caps provide no global-optimum guarantee. Priority restricts object cuts, not event cuts. Completion affects final eligibility only and cannot recover a path lost from the beam.
- **Logic:** no inferred cross-feature physics relationships, general independent multi-object quantifiers, feature decorrelation or object overlap removal. Optional pair features support two-member predicates, not arbitrary independent object quantifiers. Event-shape variables are not supplied by default.
- **Validity:** only schema-linked measurements are protected. Unguarded object eta/phi placeholders and producer-availability features need interpretation checks.
- **Zero-pass candidates:** with minimum acceptance zero, contradictory rules can enter the pool and fail logical normalization. Use a positive minimum acceptance when enabling logical pruning; the flexible runner uses 0.01.
- **Validation:** reported acceptances are training statistics; the runner does not create a validation split. Validate rules and OR menus on independent events, across resampling, mixture fractions and reconstruction/pileup conditions.
- **Rates:** no confidence-aware or combined-menu ceiling. Zero observed reference events is not zero physical rate; at 2,000 reference events a 0.1% budget permits only two observed events.
- **Deployment:** no firmware precision, latency/resource model or calibrated hardware threshold grid. Source-field limitations must be checked before interpreting detector-sensitive cuts.

Key references:

- Core code: `src/set_based_models/rule_list/`.
- Conversion and feature definitions: `raw_event_conversion/README.md`.
- Source-field caveats: `raw_event_conversion/IMPACT_PARAMETERS.md`.
- Study scoring and reporting: `casual_tests/`.